DEEP COPY VS SHALLOW COPY IN JAVA

Mental model, ownership, aliasing and production-safe choices

1 CORE MENTAL MODEL

- A variable of an object type stores a reference, not the object itself.
- Assignment copies the reference value. Both variables then point to the same object.
- A shallow copy creates a new outer object but keeps references to nested mutable objects.
- A deep copy creates independent mutable state within the copy boundary you define.
- Immutable values such as String can normally be shared safely.

2 SHALLOW COPY

- New top-level object.
- Nested mutable objects are shared.
- Faster and cheaper than deep copying.
- Changes to shared nested state can affect both copies.
- Useful when shared state is intentional or nested data is immutable.

Example

```
User copy = new User(original.id,
                     original.address);
```

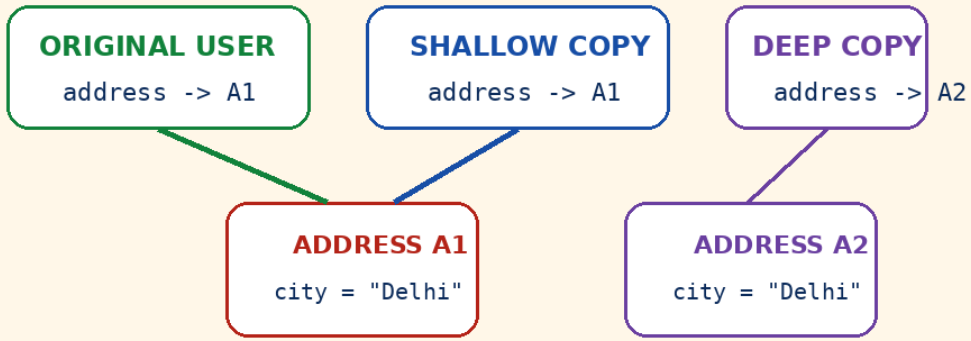
3 DEEP COPY

- New top-level object and new nested mutable objects.
- Mutating the copy does not affect the original.
- Uses more CPU and memory.
- Must handle cycles, shared identity and large graphs deliberately.
- Best for ownership boundaries, snapshots and independent processing.

Example

```
User copy = new User(original.id,
  new Address(original.address));
```

4 VISUAL REFERENCE GRAPH



A1 is shared by original and shallow copy. A2 belongs only to the deep co

5 DEEP IS A DOMAIN DECISION

- Do not blindly clone an entire object graph. Decide what the new owner must control independently.
- Resources such as sockets, database sessions, thread pools, locks and Spring services should not be deep-copied.
- Preserve shared identity only when the domain requires it. Two fields pointing to one object may need to keep that relationship.
- For immutable aggregate models, creating another copy may be unnecessary.

KEY TAKEAWAY **Assignment copies a reference; a real copy requires creating another object.**

ASSIGNMENT, FIELD MODIFICATION AND FINAL

What changes when references are assigned, mutated or shallow-copied

1 ASSIGNMENT CREATES AN ALIAS

Assignment example

```
Address a = new Address("Delhi");
Address b = a;                  // reference copied

b.setCity("Mumbai");

System.out.println(a.getCity()); // Mumbai
System.out.println(a == b);      // true
```

- No second Address is created.
- a and b are aliases for the same mutable object.
- Any mutation through either variable is visible through the other.

2 FINAL REFERENCE IS NOT IMMUTABILITY

final reference

```
final Address address = new Address("Delhi");

address.setCity("Mumbai"); // allowed
// address = new Address("Pune"); // compile error
```

- The reference stored in address cannot be replaced.
- The Address object can still change unless its API is immutable.
- final fields improve design, but deep immutability requires immutable nested state too.

3 SHALLOW COPY CONSTRUCTOR

New outer object; nested object shared

```
class User {
  private final String name;
  private final Address address;

  User(User other) {
    this.name = other.name;       // safe: String immutable
    this.address = other.address; // shared mutable reference
  }
}

User copy = new User(original);
copy.getAddress().setCity("Pune");
// original address is now also Pune
```

Use only when shared nested state is intentional or immutable.

4 DEEP COPY CONSTRUCTOR

Copy nested mutable state

```
class User {
  private final String name;
  private final Address address;

  User(User other) {
    this.name = other.name;
    this.address = new Address(other.address);
  }
}

User copy = new User(original);
copy.getAddress().setCity("Pune");
// original address remains unchanged
```

Copy constructors are explicit, testable and usually the safest default.

KEY TAKEAWAY **final protects the variable from reassignment; it does not make the referenced object immutable.**

PASSING OBJECTS AS METHOD ARGUMENTS

Java is always pass-by-value - for objects, the copied value is a reference

1 MUTATING THE OBJECT

Caller sees object mutation

```
static void rename(User user) {
  user.setName("Aisha");
}

User original = new User("Sara");
rename(original);

System.out.println(original.getName()); // Aisha
```

- The method receives its own copy of the reference.
- Both references still point to the same User.
- Calling a mutating method changes the shared object.

2 REASSIGNING THE PARAMETER

Caller reference is unchanged

```
static void replace(User user) {
  user = new User("Aisha");
}

User original = new User("Sara");
replace(original);

System.out.println(original.getName()); // Sara
```

- Only the method's local parameter is reassigned.
- The caller still holds its original reference.
- This is the clearest proof that Java is not pass-by-reference.

3 MUTABLE COLLECTION ARGUMENT

List mutation

```
static void addRole(List<String> roles) {
  roles.add("ADMIN");
}

List<String> roles = new ArrayList<>();
addRole(roles); // caller list now contains ADMIN
```

The list object is shared.

4 COPY AT THE BOUNDARY

Defensive local copy

```
static Result process(List<Order> input) {
  List<Order> working = input.stream()
      .map(Order::new) // deep copy each element
      .toList();

  // mutate working only
  return calculate(working);
}
```

Copy only when the method needs ownership.

5 PRODUCTION-SAFE API DESIGN

- Prefer immutable request objects.
- Document whether a method mutates its arguments.
- Return a new result instead of modifying input when practical.
- For performance-critical paths, avoid automatic copying; define ownership clearly.
- Never use a copy to hide unclear responsibilities.

KEY TAKEAWAY **Mutation through the parameter is visible; reassigning the parameter is not.**

RETURN VALUES AND DEFENSIVE COPYING

Protect internal state when methods return arrays, collections or mutable domain objects

1 LEAKING INTERNAL STATE

Unsafe getter

```
class Report {
  private final List<Row> rows = new ArrayList<>();

  List<Row> getRows() {
    return rows; // caller gets internal list
  }
}

report.getRows().clear(); // internal state destroyed
```

- The caller can add, remove or reorder internal data.
- This breaks encapsulation and may violate invariants.

2 RETURNING AN IMMUTABLE SNAPSHOT

Safer getter

```
List<RowView> getRows() {
  return rows.stream()
      .map(RowView::from) // immutable DTO snapshot
      .toList();          // unmodifiable list
}
```

- The list cannot be structurally modified.
- The mapped RowView values should also be immutable.
- Snapshot creation has a cost; measure on hot paths.

3 ARRAYS REQUIRE A DEFENSIVE COPY

Copy on input and output

```
class Secret {
  private final byte[] value;

  Secret(byte[] value) {
    this.value = Arrays.copyOf(value, value.length);
  }

  byte[] value() {
    return Arrays.copyOf(value, value.length);
  }
}
```

For primitive arrays, copying duplicates the values and isolates the storage.

4 VIEW VS COPY VS DEEP COPY

Technique	Structure	Elements
Collections.unmodifiableList(x)	View of x	Shared
new ArrayList<>(x)	New list	Shared
List.copyOf(x)	Unmodifiable copy*	Shared
x.stream().map(Foo::new).toList()	New list	Copied

*May reuse an existing immutable list; rejects null elements.

KEY TAKEAWAY **Returning a mutable field transfers mutation power unless you copy or expose an immutable abstraction.**

ARRAYS, COLLECTIONS AND MAPS

The container and its elements have separate copy semantics

1 PRIMITIVE ARRAY

Independent values

```
int[] a = {1, 2, 3};
int[] b = Arrays.copyOf(a, a.length);

b[0] = 99;

System.out.println(a[0]); // 1
```

- The new array has separate storage.
- Primitive values are copied.

2 OBJECT ARRAY

Array copied; elements shared

```
User[] a = { new User("A") };
User[] b = Arrays.copyOf(a, a.length);

b[0].setName("B");

System.out.println(a[0].getName()); // B
```

- The array object is new.
- Each element reference is shallow-copied.

3 LIST COPY OPTIONS

Shallow and deep element copies

```
List<User> shallow =
    new ArrayList<>(original);

List<User> immutableStructure =
    List.copyOf(original);

List<User> deep =
    original.stream()
        .map(User::new)
        .toList();
```

Only the third version copies User elements.

4 MAP COPYING

Choose key and value semantics

```
Map<String, User> shallow = new HashMap<>(original);

Map<String, User> deepValues = original.entrySet()
    .stream()
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        e -> new User(e.getValue())
    ));
```

Immutable String keys can be shared; mutable values are explicitly copied.

5 CONCURRENCY COLLECTION WARNING

- CopyOnWriteArrayList copies its internal array on writes, but it does not deep-copy the elements.
- ConcurrentHashMap provides thread-safe operations; stored mutable values can still be changed unsafely.
- A collection snapshot is stable only if its elements are immutable or independently copied.
- For concurrent workflows, immutable messages and versioned snapshots are usually easier to reason about.

KEY TAKEAWAY **A new collection does not automatically mean new element objects.**

COPY TECHNIQUES: WHAT TO USE IN PRODUCTION

Explicit code is usually safer than generic graph cloning

1 COPY CONSTRUCTOR / FACTORY

- Explicit and type-safe.
- Lets you choose deep vs shared fields.
- Can reset IDs, versions or audit fields.
- Easy to unit-test and review.

Recommended

```
Order copy = Order.copyOf(original);
```

2 BUILDER / WITHER

- Good for immutable objects.
- Makes changed fields visible at call sites.
- Nested mutable values still need explicit copying.
- Lombok toBuilder() is shallow unless you replace nested fields.

Example

```
Order revised = original.toBuilder()
    .status(APPROVED)
    .build();
```

3 RECORDS

- Record components are final, but referenced objects may still be mutable.
- A record is shallowly immutable, not automatically deeply immutable.
- Use immutable components or copy mutable inputs in the canonical constructor.

Caution

```
record Team(List<User> members) { }
// members can still reference a mutable list
```

4 OBJECT.CLONE()

- Object.clone() performs a field-by-field shallow copy.
- Requires Cloneable and awkward exception handling.
- Constructors are not called.
- Easy to forget nested mutable state.
- Usually avoid in new production code.

5 SERIALIZATION / JSON ROUND TRIP

- Convenient for prototypes or rare operations.
- Slower and allocation-heavy.
- Can lose subtype, transient, identity or precision information.
- Java serialization has security and versioning concerns.
- Do not use as the default deep-copy strategy.

6 GENERATED MAPPING

- MapStruct can generate explicit field mappings at compile time.
- Useful for Entity -> DTO, API snapshots and copy boundaries.
- Nested mappings must still be configured deliberately.
- Business rules and database calls should remain outside the mapper.

KEY TAKEAWAY **Prefer the smallest explicit copy that communicates ownership and preserves invariants.**

PRODUCTION SCENARIOS AND RECOMMENDED CHOICES

Where accidental sharing causes real defects

Scenario	Recommended approach	Production caution
API request input	Immutable DTO or validated defensive copy	Do not let controller input mutate domain state directly
API response	Immutable response DTO snapshot	Returning entities can leak lazy proxies and internal fields
Cache value	Immutable value or deep snapshot	Caller mutation can corrupt later cache hits
Async task / event	Immutable message or copied payload	Publisher and consumer may run after source changes
JPA/Hibernate entity	Explicit DTO/factory; reset id/version	clone may copy identity, proxies, cycles and lazy associations
Audit/history	Immutable version snapshot	Store only required state; large graphs are expensive
Concurrent algorithm	Per-worker copy or immutable shared input	Shallow nested state can create race conditions

1 CACHE EXAMPLE

Cache immutable DTOs, not mutable entities.

2 EVENT EXAMPLE

Publish a snapshot that cannot change later.

3 ORM EXAMPLE

Map managed entities to purpose-built copies.

KEY TAKEAWAY **Copy at ownership boundaries - not automatically at every method call.**

DECISION GUIDE, TESTS AND INTERVIEW REVISION

A last-minute checklist for choosing the right copy semantics

1 DECISION GUIDE

- Will the receiver mutate the object? If no, prefer immutable sharing.
- Must mutations remain independent? Copy the mutable state inside that ownership boundary.
- Is the graph large or cyclic? Avoid generic deep cloning; design a smaller snapshot.
- Are IDs, versions, secrets or lazy proxies present? Use an explicit mapper or factory.
- Is this a hot path? Benchmark allocation rate, latency and GC impact.
- Can the contract state ownership instead of copying? Clear APIs often remove the need.

2 UNIT TEST THE COPY CONTRACT

JUnit example

```
User original = sampleUser();
User copy = new User(original);

assertNotSame(original, copy);
assertNotSame(original.getAddress(), copy.getAddress());

copy.getAddress().setCity("Pune");

assertEquals("Delhi",
             original.getAddress().getCity());
assertEquals("Pune",
             copy.getAddress().getCity());
```

Also test collections, nulls, cycles and identity-sensitive relationships.

3 30-SECOND INTERVIEW ANSWERS

- Assignment does not copy an object; it copies the reference value.
- Java always passes arguments by value. Object mutations can be visible because both references point to the same object.
- A shallow copy duplicates the outer object but shares nested references.
- A deep copy duplicates the mutable state required for independent ownership.
- List.copyOf makes the list unmodifiable, but it does not deep-copy the elements.
- Object.clone() is shallow by default and is usually avoided in modern production code.

4 FINAL PRODUCTION CHECKLIST

- Copy boundary documented.
- Mutable nested fields identified.
- IDs, versions and audit fields handled intentionally.
- Collections and elements copied at the correct depth.
- No accidental copying of services or resources.
- Tests prove independent mutation where required.
- Performance measured under realistic load.

KEY TAKEAWAY **The correct answer is not always deep copy - it is explicit ownership with tested behavior.**